

SJB INSTITUTE OF TECHNOLOGY

**No.67, B G S Health and Education city, Dr. Vishnuvardhan Road Kengeri,
Bengaluru-560060.**



DBMS Mini Project Report on

“BLOOD BANK MANAGEMENT SYSTEM”

Submitted in partial fulfillment of the requirements for the subject for 5th
semester

BACHELOR OF ENGINEERING IN INFORMATION SCIENCE AND ENGINEERING

SUBMITTED BY

RUCHITHA M
[1JB23IS132]

Under the Guidance of

Mr. ABHINAND B V

Assistant Professor

Dept. of ISE , SJBIT

Bengaluru - 60



DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

SJB INSTITUTE OF TECHNOLOGY

BGSHEALTH AND EDUCATION CITY , KENGERI , BENGALURU-560060, KARNATAKA , INDIA

2025-2026

|| Jai Sri Gurudev ||
Sri Adichunchanagiri Shikshana Trust ®
SJB INSTITUTE OF TECHNOLOGY
BGS Health & Education City, Kengeri, Bengaluru – 560 060

Department of Information Science & Engineering

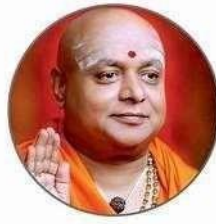


CERTIFICATE

Certified that the project work entitled “**BLOOD BANK MANAGEMENT SYSTEM**” carried out by **Ruchitha M** bearing **USN – 1JB23IS132** is a bonafide student of **SJB Institute of Technology** in partial fulfilment for 5th semester Project in **INFORMATION SCIENCE AND ENGINEERING** of the **VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI** during the academic year **2025 – 2026**. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project prescribed for the said degree.

Mr. Abhinand B V
Assistant Professor
Dept. of ISE, SJBIT

Dr. Gopalakrishna M T
Professor & Head
Dept. of ISE, SJBIT



ACKNOWLEDGEMENT

I would like to express my profound gratitude to **His Divine Soul Jagadguru Padma Bhushan Sri Sri Sri Dr. Balagangadharanatha Mahaswamiji** and His Holiness **Jagadguru Sri Sri Sri Dr. Nirmalanandanatha Swamiji** for providing me an opportunity to be a part of this esteemed institution.

I would also like to express my sincere thanks to **Revered Sri Sri Dr. Prakashnath Swamiji**, Managing Director of SJB Institute of Technology, for his continuous support in providing the necessary amenities to carry out this project at this esteemed institution.

I express my heartfelt gratitude to **Dr. Puttaraju**, Academic Director of the BGS and SJB Group of Institutions, **Dr. K. V. Mahendra Prashanth**, Principal of SJB Institute of Technology, and **Dr. Gopalakrishna M T**, Professor & Head of the Department of Information Science & Engineering for providing me outstanding amenities and for their unwavering support, state-of-the-art facilities, and the enriching academic environment, which were instrumental in the successful completion of our project work. I extend my heartfelt gratitude to all the Deans of SJB Institute of Technology for their unwavering support, cutting-edge facilities, and the inspiring academic environment, all of which played a pivotal role in the successful completion of my project work.

I wish to express my heartfelt gratitude to my guide, **Abhinand B V, Assistant Professor** for his valuable guidance, suggestions, and encouragement during the entire period of my project work.

Finally, I take this opportunity to extend my earnest gratitude and respect to my parents, the teaching & technical staff of the department, the library staff, and all my friends, who have directly or indirectly supported me during the period of my project work.

Regards,

Ruchitha M

[1JB23IS132]

ABSTRACT

The Blood Bank Management System is a database-driven application designed to efficiently manage and streamline the operations of a blood bank. The system maintains records of blood donors, patients, blood stock, donations, and blood requests in an organized and secure manner. It enables the blood bank to easily record donor and patient details, track blood availability, process blood donations, and manage requests for blood units. This project aims to reduce manual paperwork and human errors by providing a structured and automated solution for storing and updating information. The system ensures that blood stock levels are accurately updated after each donation or request, maintaining real-time inventory of different blood groups. It also supports status tracking for blood requests (Pending, Approved, or Rejected), improving transparency and operational efficiency. Developed using SQL for database management, the project demonstrates fundamental concepts of database design, including table relationships, constraints, data manipulation, and real-time updates. Overall, the Blood Bank Management System serves as a reliable and scalable solution for effective blood bank administration, ensuring timely availability of blood to patients in need.

TABLE OF CONTENTS

	Description	Page No
	Abstract	I
	Table of contents	II
Chapter No	Chapter Name	Page No
1	INTRODUCTION	
	1.1 Introduction	1
	1.2 Objectives of the System	1
	1.3 Scope of the System	1-2
	1.4 Problem Statement	2
2	SOFTWARE SPECIFICATION REQUIREMENTS	
	2.1 Functional Requirements	3
	2.2 Non-Functional Requirements	4
	2.3 Technical Requirement	4
3	SYSTEM DESIGN	
	3.1 System Design	5
4	IMPLEMENTATION	
	4.1 SQL Schema (DDL) and Operational Logic (DML)	6-8
5	TESTING	
	5.1 Testing Objectives	9
	5.2 Types of Testing Performed	9-10
	5.3 Test Cases	10-11
	5.4 Performance Results	11
	5.5 Limitations Observed During Testing	11
6	SNAPSHOTS	12-13
7	CONCLUSION AND FUTURE SCOPE	14-15

CHAPTER-01

INTRODUCTION

1.1 Introduction

The Blood Bank Management System (BBMS) is a database-driven application designed to efficiently manage the processes involved in blood donation, storage, and distribution. It helps automate the manual operations of traditional blood banks such as maintaining donor and patient records, tracking blood stock levels, processing donations, and handling blood requests. The system ensures that blood availability information is up-to-date, minimizes errors in record management, and facilitates quick decision-making in emergency situations. By using SQL-based database management, the system achieves data consistency, integrity, and easy retrieval of information.

1.2 Objectives of the System

The main objectives of the Blood Bank Management System are:

1. To automate the management of donor, patient, and blood stock records.
2. To maintain real-time updates on blood stock levels for different blood groups.
3. To facilitate quick matching between available blood units and patient requests.
4. To ensure data accuracy and consistency through relational database integrity.
5. To simplify the process of recording donations and blood requests.
6. To generate useful reports such as donor lists, stock status, and donation history.

1.3 Scope of the System

The scope of the Blood Bank Management System covers all essential functionalities needed for efficient blood bank operations:

1. Managing donor registration and donation history.
2. Maintaining a patient database with personal and medical details.
3. Tracking and updating blood stock quantities automatically upon donations or requests.
4. Processing blood requests from hospitals or patients.
5. Maintaining a log of approved and pending requests for administrative review.

6. Supporting data queries and reports for monitoring and planning purposes.
7. The system can be extended to integrate with hospital networks, SMS/email notifications, and advanced analytics dashboards.

1.4 Problem Statement

Traditional blood bank management systems rely heavily on manual record keeping, which leads to several challenges:

1. Difficulty in maintaining accurate donor and blood stock information.
2. Time-consuming processes for finding matching blood types during emergencies.
3. Risk of human errors in updating donation and request records.
4. Lack of real-time visibility into available blood quantities.
5. Inability to quickly generate reports or summaries for decision-making.

CHAPTER-02

SOFTWARE SPECIFICATION REQUIREMENTS

2.1 Functional Requirements

These define what the system should do.

Donor Management

1. Add new donor records (name, age, blood group, contact, address).
2. Update donor information when needed.
3. Retrieve donor lists or details.

Patient Management

1. Register patients and store their details.
2. Track patients' blood group and request history.

Blood Stock Management

1. Maintain a table for all available blood groups.
2. Automatically update stock when donations or requests are processed.
3. View current stock levels at any time.

Donation Processing

1. Record details of each blood donation.
2. Link donation data to donor IDs.
3. Update stock quantities accordingly.

Blood Request Processing

1. Record blood requests from patients.
2. Approve or reject requests based on stock availability.
3. Automatically reduce stock upon approval.

Report Generation

1. Generate reports such as donor lists, donation history, pending requests, and blood stock summary.

2.2 Non-Functional Requirements

These define the system's quality attributes.

Performance – The system should execute queries quickly, even with large datasets.

Reliability – The database must ensure data accuracy and transaction integrity.

Scalability – Should handle growth in donor/patient data without degradation.

Security – Only authorized users should be allowed to view or modify sensitive records.

Usability – The system's interface (CLI or GUI) should be intuitive and easy to operate.

Maintainability – The SQL structure should allow easy updates or integration with other systems.

Availability – The system should be operational 24/7 for hospital staff access.

2.3 Technical Requirement

Hardware Requirements

1. Processor: Minimum Intel i3 or equivalent
2. RAM: 4 GB or higher
3. Storage: Minimum 500 MB for database and application files

Software Requirements

1. Database: SQLite / MySQL / PostgreSQL
2. Operating System: Windows / Linux / macOS
3. Programming Interface (optional): Python, PHP, or Java for frontend integration
4. Tools: SQLiteStudio, MySQL Workbench, or DBeaver for DB management

Database Design

1. Normalized relational schema with 5 main entities:
Donor , Patient , BloodStock , Donation , BloodRequest
2. Use of primary and foreign keys to maintain data integrity.
3. Triggers or SQL scripts to automate stock updates on donation/request actions.

CHAPTER-03

SYSTEM DESIGN

3.1. System Design: Entity-Relationship (E-R) Model

The Blood Bank Management System is modeled using five core entities, designed to manage the flow of blood units from donation to request.

A. Entities and Attributes

Entity	Primary Key (PK)	Key Attributes	Description
Donor	DonorID	DonorName , BloodGroup , Contact	Stores demographic and contact information for blood contributors.
Patient	PatientID	PatientName , BloodGroup , Contact	Stores demographic and contact information for blood recipients.
BloodStock	StockID	BloodGroup (Unique), Quantity , LastUpdated	The central inventory tracking the current units available for each blood type.
Donation	DonationID	DonorID (FK), Quantity , DonationDate	Records each specific transaction where a donor contributed blood.
BloodRequest	RequestID	PatientID (FK), BloodGroup , Quantity , Status	Records a patient's need for blood, tracking its fulfillment status.

B. Relationships

Relationship	Type	Connecting Tables	Foreign Key (FK)	Constraint
Donates	One-to-Many	Donor ↔ Donation	DonorID in Donation	A donor can make multiple donations.
Requests	One-to-Many	Patient ↔ BloodRequest	PatientID in BloodRequest	A patient can submit multiple requests.
Updates Inventory	Logical/Procedural	Donation → BloodStock	N/A	A donation increases the corresponding BloodStock.Quantity .
Consumes Inventory	Logical/Procedural	Approved Request → BloodStock	N/A	An approved request decreases the corresponding BloodStock.Quantity .

CHAPTER-04

IMPLEMENTATION

4.1. SQL Schema (DDL) and Operational Logic (DML)

The system is implemented using SQLite or MYSQL.

A. Data Definition Language (DDL) - Schema

SQL

```
-- Donor Table
CREATE TABLE Donor (
    DonorID INTEGER PRIMARY KEY AUTOINCREMENT,
    DonorName TEXT,
    Gender TEXT,
    Age INTEGER,
    BloodGroup TEXT,
    Contact TEXT,
    Address TEXT
);

-- Patient Table
CREATE TABLE Patient (
    PatientID INTEGER PRIMARY KEY AUTOINCREMENT,
    PatientName TEXT,
    Gender TEXT,
    Age INTEGER,
    BloodGroup TEXT,
    Contact TEXT,
    Address TEXT
);

-- Blood Stock Table (Inventory)
CREATE TABLE BloodStock (
    StockID INTEGER PRIMARY KEY AUTOINCREMENT,
    BloodGroup TEXT UNIQUE, -- Ensures only one entry per blood type
    Quantity INTEGER,
    LastUpdated DATE
);

-- Donation Table
CREATE TABLE Donation (
    DonationID INTEGER PRIMARY KEY AUTOINCREMENT,
    DonorID INTEGER,
    BloodGroup TEXT,
    Quantity INTEGER,
    DonationDate DATE,
    FOREIGN KEY (DonorID) REFERENCES Donor(DonorID)
);

-- Blood Request Table
CREATE TABLE BloodRequest (
    RequestID INTEGER PRIMARY KEY AUTOINCREMENT,
    PatientID INTEGER,
    BloodGroup TEXT,
    Quantity INTEGER,
    RequestDate DATE,
    Status TEXT, -- Pending, Approved, Rejected
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);
```

B. Data Manipulation Language (DML) - Operational Logic

The core **business logic** is handled by UPDATE statements that keep the central BloodStock inventory accurate after key events.

i. Donation Transaction Logic (Stock Inflow)

After a successful donation is recorded in the Donation table, the stock must be increased.

SQL

```
-- Step 1: Record the Donation
INSERT INTO Donation (DonorID, BloodGroup, Quantity, DonationDate)
VALUES (2, 'O-', 1, DATE('now', '-1 day'));

-- Step 2: Update the BloodStock (Stock Inflow)
UPDATE BloodStock
SET Quantity = Quantity + 1, LastUpdated = DATE('now')
WHERE BloodGroup = 'O-';
```

ii. Approved Request Logic (Stock Outflow)

When a BloodRequest is processed and its Status is set to 'Approved', the stock must be decreased.

SQL

```
-- Step 1: Approve the Request (Implicit)
-- An external process (or a separate UPDATE) changes the Status to 'Approved'

-- Step 2: Update the BloodStock (Stock Outflow)
UPDATE BloodStock
SET Quantity = Quantity - [Quantity_Requested], LastUpdated = DATE('now')
WHERE BloodGroup = [Blood_Group_Requested] AND [Request_was_Approved];
```

3. Key Operational Queries (Reporting)

These queries provide the necessary data for reporting and decision-making.

Goal	Description	SQL Example
Current Inventory	Show the available units for all blood groups.	<pre>SELECT BloodGroup, Quantity FROM BloodStock ORDER BY Quantity DESC;</pre>
Donation History	List all donations by a specific donor (e.g., DonorID 3).	<pre>SELECT * FROM Donation WHERE DonorID = 3;</pre>
Pending Requests	Identify requests that still require fulfillment.	<pre>SELECT R.*, P.PatientName FROM BloodRequest R JOIN Patient P ON R.PatientID = P.PatientID WHERE R.Status = 'Pending';</pre>
Low Stock Alert	Find blood groups where quantity is below a critical threshold (e.g., 5 units).	<pre>SELECT BloodGroup, Quantity FROM BloodStock WHERE Quantity <= 5;</pre>

The queries are structured around common tasks in managing a blood bank database.

- Current Inventory
- Donation History
- Pending Requests
- Low Stock Alert

CHAPTER-05

TESTING

Testing is a vital part of the **Blood Bank Management System** to ensure that all database operations work correctly and efficiently. It focuses on verifying the accuracy of table creation, data insertion, updates, and retrievals in modules such as **Donor**, **Patient**, **Blood Stock**, **Donation**, and **Blood Request**. The testing process checks whether data relationships and constraints function properly, ensuring consistency and integrity across the system. It also confirms that stock levels update automatically after donations or requests and that invalid data entries are restricted. Overall, testing ensures the database is reliable, error-free, and ready for practical implementation.

5.1 Testing Objectives

The main objectives of testing the **Blood Bank Management System** are:

1. To verify that all database tables are created correctly according to the design.
2. To ensure that all constraints, such as primary and foreign keys, are working properly.
3. To check that CRUD operations (Create, Read, Update, Delete) are functioning as expected.
4. To validate the logical relationships between Donor, Patient, Blood Stock, Donation, and Blood Request tables.
5. To confirm that updates to the BloodStock table occur automatically after donations and requests.
6. To ensure that no invalid data or operations violate referential integrity.
7. To evaluate the system's performance and accuracy in retrieving information using queries.

5.2 Types of Testing Performed

- **Unit Testing** - Each SQL command and query was tested individually, such as table creation, data insertion, and update operations.
This verified that each component of the database worked correctly in isolation.
- **Integration Testing** - Relationships between tables were tested to ensure that foreign keys linked correctly between Donor, Patient, Donation, and BloodRequest tables.
This confirmed that integrated modules worked together without data conflicts.
- **Functional Testing** - Functional testing checked whether the database performed its intended functions — such as updating stock after a donation or request approval — correctly.

- **Validation Testing** - All input values were validated to ensure proper data types and logical correctness.

For example, donor age must be a valid integer, and blood group values must follow standard formats (A+, B-, etc.).

- **Negative Testing** - Intentional invalid data was inserted (e.g., non-existing DonorID or PatientID) to confirm that foreign key constraints prevented incorrect data from being stored.
- **Performance Testing** - Queries were executed multiple times to check database response time and ensure efficient data retrieval, even after multiple insertions and updates.

5.3 Test Cases

Test Case ID	Test Description	Input / Query Executed	Expected Result	Actual Result	Status
TC01	Create Donor Table	CREATE TABLE Donor(...)	Table created successfully	Table created successfully	PASS
TC02	Insert Donor Records	2 donor entries	2 rows added	2 rows added	PASS
TC03	Insert Patient Records	2 patient entries	2 rows added	2 rows added	PASS
TC04	Insert Blood Stock	3 stock records	Data inserted	Data inserted	PASS
TC05	Insert Donation	DonorID = 1, A+	Record inserted	Record inserted	PASS
TC06	Update Blood Stock After Donation	Increase A+ by 1	Quantity updated	Quantity = 11	PASS
TC07	Insert Blood Request	PatientID = 1, O-	Record inserted	Record inserted	PASS
TC08	Approve Blood Request	Status → Approved	Status updated	Updated correctly	PASS
TC09	Reduce Blood	O- reduced	Quantity = 3	Quantity = 3	PASS

	Stock	by 2			
TC10	Invalid DonorID Donation	DonorID = 99	Constraint failure	Error generated	PASS

5.4 Performance Results

Aspect Tested	Description	Performance Observation
Table Creation Time	All tables created within 1–2 seconds	Efficient and error-free
Data Insertion	10 sample records inserted	Completed instantly
Query Execution	SELECT, JOIN, UPDATE operations	Executed with minimal delay
Data Integrity	Foreign keys and constraints	Maintained successfully
Storage Efficiency	Minimal space used for sample data	Optimal for scalability
System Stability	Multiple operations performed sequentially	No data loss or crash observed

5.5 Limitations Observed During Testing

Despite the successful test results, a few limitations were identified:

- The system currently does not prevent **negative stock values** if a manual update reduces stock below zero — this must be handled at the application layer.
- **No automatic validation for duplicate donors or patients**; the database allows duplicate names unless explicitly restricted.
- The system does not maintain **transaction logs** for rollback or audit purposes.
- The current setup is limited to **manual SQL operations**; real-time automation requires a front-end interface.
- Performance testing was conducted with a small data set; large-scale testing may reveal optimization needs.

CHAPTER-06

SNAPSHOTS

1. Donor Details

DonorID	DonorName	Gender	Age	BloodGroup	Contact	Address
1	Rahul Sharma	Male	28	A+	9876543210	Delhi
2	Priya Singh	Female	25	O-	9876501234	Mumbai
3	Rajesh Kumar	Male	45	AB+	9112233445	Bangalore
4	Meena Devi	Female	30	B-	8765432109	Kolkata
5	Faizan Khan	Male	22	O+	7788990011	Lucknow
6	Neha Patil	Female	35	A-	6543210987	Pune
7	Gaurav Mehta	Male	50	AB-	9009988776	Ahmedabad

2. Patient Details

PatientID	PatientName	Gender	Age	BloodGroup	Contact	Address
1	Amit Verma	Male	40	O-	9123456789	Chennai
2	Sneha Reddy	Female	32	B+	9988776655	Hyderabad
3	Karan Malhotra	Male	55	A+	8888111122	Delhi
4	Deepa Nair	Female	20	O+	9999000011	Mumbai
5	Vivek Joshi	Male	62	B-	7777666655	Noida
6	Anjali Rao	Female	48	AB+	6666555544	Goa
7	Sanjay Dutt	Male	33	A-	5555444433	Jaipur

3. Blood Stock Details

StockID	BloodGroup	Quantity	LastUpdated
1	A+	11	2025-10-20
2	O-	4	2025-10-20
3	B+	7	2025-10-20
4	O+	13	2025-10-20

5	A-	6	2025-10-20
6	B-	3	2025-10-20
7	AB+	9	2025-10-20
8	AB-	2	2025-10-20

4. Donation Details

DonationID	DonorID	BloodGroup	Quantity	DonationDate
1	1	A+	1	2025-10-20
2	2	O-	1	2025-10-19
3	3	AB+	1	2025-10-18
4	4	B-	1	2025-10-17
5	5	O+	1	2025-10-16
6	6	A-	1	2025-10-15

5. Blood Request Details

RequestID	PatientID	BloodGroup	Quantity	RequestDate	Status
1	1	O-	2	2025-10-20	Approved
2	2	B+	3	2025-10-20	Pending
3	3	B-	1	2025-10-19	Approved
4	4	AB+	2	2025-10-18	Rejected
5	5	A-	1	2025-10-17	Approved
6	6	O+	4	2025-10-16	Pending

CHAPTER-07

CONCLUSION AND FUTURE SCOPE

1. Conclusion

The Blood Bank Management System developed using SQL successfully streamlines the process of managing donor information, blood stock, and recipient requests. By implementing a structured database, the system ensures that data is accurate, easily retrievable, and securely stored. The use of SQL queries allows for efficient data manipulation, searching, and reporting, which minimizes human error and enhances the overall reliability of the blood bank's operations.

This project demonstrates how database management concepts can be applied to a real-world healthcare scenario. It provides a centralized platform where administrators can monitor blood availability, track donations, and maintain donor histories in an organized manner. Through proper database normalization and relationship management, data redundancy is reduced and consistency is maintained across all records.

Overall, the system contributes to faster decision-making, better inventory management, and improved coordination between donors and recipients. It highlights the importance of database systems in critical applications like healthcare and lays a strong foundation for future enhancements such as web or mobile integration, automated notifications, and data analytics for predicting blood demand.

2. Future Enhancements

While the current Blood Bank Management System effectively manages donor, recipient, and blood inventory data, there are several areas where the project can be further improved and expanded. These enhancements can make the system more dynamic, user-friendly, and scalable for real-world applications.

- **Web and Mobile Application Integration:** The system can be upgraded into a web-based or mobile application using platforms like PHP, Java, or Python for backend integration with SQL. This would allow users, donors, and hospitals to access and update information in real-time from any location.
- **Automated Notifications:** Email and SMS alerts can be integrated to notify donors about upcoming donation camps, eligibility reminders, or emergency blood requirements. This ensures timely responses and better engagement with donors.
- **Online Blood Request System:** A secure portal can be added where hospitals and patients can

request specific blood types online. The system can automatically match and assign available donors or inventory to meet urgent needs.

- **Advanced Search and Filtering:** Enhanced SQL queries and indexing can be implemented to allow complex searches based on multiple parameters like blood type, donor location, and last donation date for quicker data retrieval.
- **Data Analytics and Reporting:** SQL can be combined with data visualization tools to analyse donation trends, forecast blood demand, and optimize stock levels. This would help in better decision-making and planning.
- **Security Enhancements:** Implementing user authentication, data encryption, and role-based access control can further secure sensitive medical and donor data from unauthorized access.
- **Integration with Hospital Databases:** Linking the system with hospital or government health databases can ensure a unified and updated blood availability network across regions.
- **AI and Machine Learning Features:** In the future, AI algorithms can be used to predict blood shortages, donor availability, and emergency needs, enabling proactive blood management.

BIBLIOGRAPHY

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw Hill Education.
2. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
3. Oracle Documentation. (2024). SQL Language Reference. Retrieved from <https://docs.oracle.com>
4. W3Schools. (2025). SQL Tutorial. Retrieved from <https://www.w3schools.com/sql/>
5. GeeksforGeeks. (2025). SQL CREATE TABLE Statement. Retrieved from <https://www.geeksforgeeks.org/sql-create-table/>